

2023년 2회 정보처리기사 실기 복원 문제 (해설 추가)

출처: <https://chobopark.tistory.com/420> (Life-Journey)

1번

다음은 C언어 코드의 문제이다. 보기의 조건에 맞도록 괄호 안에 알맞은 코드를 작성하시오.

보기: 입력값이 54321일 경우 출력값이 43215로 출력되어야 한다.

```
int main(void) {
    int n[5];
    int i;

    for (i = 0; i < 5; i++) {
        printf("숫자를 입력해주세요 : ");
        scanf("%d", &n[i]);
    }

    for (i = 0; i < 5; i++) {
        printf("%d", (
            ) );
    }

    return 0;
}
```

정답: `n[(i+1) % 5]`

해설:

배열의 값을 **한 칸씩 앞으로 당겨서(회전시켜)** 출력하는 문제입니다.

입력: `n[0]=5, n[1]=4, n[2]=3, n[3]=2, n[4]=1` (54321)
원하는 출력: 4 3 2 1 5 → 즉 두 번째 값부터 출력하고 첫 값은 맨 뒤로 보냄.

- `i` 가 0,1,2,3,4로 돌 때 출력해야 할 인덱스는 1,2,3,4,**0**입니다.
- `(i+1)` 을 하면 1,2,3,4,5가 되는데, 마지막 5는 배열 범위를 벗어납니다(인덱스는 0~4).
- 여기서 **나머지 연산 % 5** 를 씁니다. `5 % 5 = 0` 이 되어 다시 처음으로 돌아옵니다.
- 따라서 `n[(i+1) % 5]` → 인덱스 1,2,3,4,0 → 값 4,3,2,1,5 → "43215" 출력

핵심 패턴: **% 배열크기** 는 **인덱스를 빙글빙글 돌리는(원형) 데 쓰입니다**. 큐, 원형버퍼 등에서 자주 나옵니다.

2번

다음은 JAVA 코드 문제이다. 가지고 있는 돈이 총 4620원일 경우 1000/500/100/10원 지폐·동전을 이용하여 괄호 안을 작성하시오.

보기: 변수 `m` / 연산자 `/`, `%` / 괄호 `[]()` / 정수 1000, 500, 100, 10

```
public class Problem {
    public static void main(String[] args){
        m = 4620;

        a = (
        );
        b = (
        );
        c = (
        );
        d = (
        );

        System.out.println(a); // 천원짜리    4장 출력
        System.out.println(b); // 오백원짜리  1개 출력
        System.out.println(c); // 백원짜리    1개 출력
        System.out.println(d); // 십원짜리    2개 출력
    }
}
```

```
}  
}
```

정답:

- a = m / 1000
- b = (m % 1000) / 500
- c = (m % 500) / 100
- d = (m % 100) / 10

해설:

거스름돈 계산의 전형적인 문제로, 나눗셈(/)으로 개수를 구하고 나머지(%)로 남은 돈을 구하는 패턴입니다.

정수 나눗셈에서는 소수점이 버려진다는 점이 핵심입니다.

- a = m / 1000 → 4620 / 1000 = 4 (천원짜리 4장)
- 남은 돈: m % 1000 = 4620 % 1000 = 620
- b = (m % 1000) / 500 → 620 / 500 = 1 (오백원 1개)
- 남은 돈: m % 500 = 4620 % 500 = 120
- c = (m % 500) / 100 → 120 / 100 = 1 (백원 1개)
- 남은 돈: m % 100 = 4620 % 100 = 20
- d = (m % 100) / 10 → 20 / 10 = 2 (십원 2개)

핵심 규칙: "큰 단위로 나눠서 개수 구하고, 그 단위로 나눈 나머지에서 다음 단위를 처리"하는 반복 패턴입니다.

3번

다음은 C언어의 코드이다. 보기의 조건에 맞추어 알맞은 출력값을 작성하시오.

보기: 입력값은 홍길동, 김철수, 박영희 순서로 주어진다.

```
#include <stdio.h>  
#include <stdlib.h>  
char n[30];  
  
char *test() {  
    printf("입력하세요 : ");  
    gets(n);  
    return n;  
}  
  
int main() {  
    char * test1;  
    char * test2;  
    char * test3;  
  
    test1 = test();  
    test2 = test();  
    test3 = test();  
  
    printf("%s\n", test1);  
    printf("%s\n", test2);  
    printf("%s", test3);  
}
```

정답: 박영희 / 박영희 / 박영희

해설:

전역 변수와 포인터의 함정을 보여주는 문제입니다. 핵심은 char n[30] 이 함수 밖에 선언된 전역 배열이라는 점입니다.

- test() 함수는 매번 같은 전역 배열 n 에 입력을 받고, n 의 주소를 반환합니다.
- test1, test2, test3 는 모두 같은 주소(n의 주소)를 가리키게 됩니다. 서로 다른 값을 복사해 저장하는 게 아니라, 셋 다 같은 메모리를 바라봅니다.
- 입력은 홍길동 → 김철수 → 박영희 순으로 들어오는데, 매번 같은 n 에 덮어쓰기 됩니다. 마지막 입력 "박영희"가 n 에 남습니다.
- 결국 test1, test2, test3 모두 같은 n ("박영희")을 가리키므로 셋 다 "박영희" 출력.

핵심 교훈: 포인터가 같은 메모리를 가리키면, 마지막에 저장된 값만 남습니다. 각각 따로 저장하려면 매번 새 메모리를 할당해 복사해야 합니다.

4번

다음은 테이블에 데이터를 삽입하기 위한 과정이다. 보기의 조건식에 맞게 SQL문을 작성하시오.

보기: 학생 테이블 [학번 int, 이름 varchar(20), 학년 int, 전공 varchar(30), 전화번호 varchar(20)]

결과값: 9830287, 뉴진스, 3, 경영학개론, 010-1234-1234

정답:

```
INSERT INTO 학생 (학번, 이름, 학년, 전공, 전화번호)
VALUES (9830287, '뉴진스', 3, '경영학개론', '010-1234-1234');
```

해설:

새로운 데이터(행)를 테이블에 추가할 때는 INSERT INTO 문을 씁니다. 기본 형식:

```
INSERT INTO 테이블명 (열1, 열2, ...) VALUES (값1, 값2, ...);
```

주의할 점:

- 열의 순서와 값의 순서가 일치해야 합니다.
- 문자(varchar) 타입 값은 작은따옴표(")로 감쌉니다. → '뉴진스', '경영학개론', '010-1234-1234'
- 숫자(int) 타입 값은 따옴표 없이 그대로 씁니다. → 9830287, 3
- 전화번호는 varchar(문자)이므로 숫자처럼 보여도 반드시 따옴표를 붙입니다.

핵심: "문자는 따옴표 O, 숫자는 따옴표 X"가 가장 흔한 실수 포인트입니다.

5번

다음은 C언어의 문제이다. 알맞은 출력값을 작성하시오.

```
#include <stdio.h>

void main(){
    int n[3] = {73, 95, 82};
    int sum = 0;

    for(int i=0; i<3; i++){
        sum += n[i];
    }

    switch(sum/30){
        case 10:
        case 9: printf("A");
        case 8: printf("B");
        case 7:
        case 6: printf("C");
        default: printf("D");
    }
}
```

정답: BCD

해설:

switch문에서 break가 없을 때 발생하는 "폴스루(fall-through)" 현상을 묻는 문제입니다.

먼저 계산:

- sum = 73 + 95 + 82 = 250
- sum / 30 = 250 / 30 = 8 (정수 나눗셈이므로 소수점 버림)

이제 switch(8) 이므로 case 8: 부터 실행합니다. break가 없으면 그 아래 case들이 줄줄이 실행됩니다.

- case 8: → "B" 출력 (break 없음 → 계속)
- case 7: (실행할 게 없음) → case 6: → "C" 출력 (break 없음 → 계속)
- default: → "D" 출력

결과: BCD

핵심: switch문에서 **break가 없으면 만난 case 이후 모든 코드가 실행**됩니다. break를 만나거나 switch가 끝나야 멈춥니다. 이 "break 누락" 함정은 시험 단골입니다.

6번

다음은 테스트 커버리지에 대한 내용이다. 보기에서 알맞는 기호를 고르시오.

- 프로그램 내 결정포인트 내의 모든 개별 조건식에 대한 모든 가능한 결과(참/거짓)를 적어도 한 번 수행
- True/False에 충분한 영향을 줄 수 없는 경우가 발생 가능한 한계점을 지님

보기: ㄱ. 구문 ㄴ. 경로 ㄷ. 조건/결정 ㄹ. 변형 조건/결정 ㄴ. 다중 조건 ㄷ. 결정 ㄹ. 조건

정답: ㄹ (조건 커버리지)

해설:
조건 커버리지(Condition Coverage)를 묻는 문제입니다.

조건문이 if (A && B) 처럼 여러 개의 작은 조건(A, B)으로 이루어졌다고 합시다.

- 조건 커버리지는 각각의 개별 조건(A, B)이 참(True)도 한 번, 거짓(False)도 한 번 되도록 테스트하는 기법입니다.
- 하지만 개별 조건만 참/거짓을 맞추다 보면, 전체 결정(if문 최종 결과)은 한쪽만 테스트될 수도 있는 한계가 있습니다. 지문의 "True/False에 충분한 영향을 줄 수 없는" 부분이 이 한계를 말합니다.

- 커버리지 종류 비교:
- 결정(분기) 커버리지: if문의 전체 결과(T/F)를 모두 테스트
 - 조건 커버리지: if문 안의 개별 조건 각각의 T/F를 모두 테스트
 - 조건/결정 커버리지: 위 둘을 모두 만족

핵심 구분: "개별 조건식의 참/거짓"이라는 표현이 나오면 조건 커버리지입니다.

7번

다음 소스코드의 알맞은 출력을 작성하십시오.

```
#include <stdio.h>

int main(){
    int c = 0;
    for(int i = 1; i <= 2023; i++) {
        if(i%4 == 0) c++;
    }
    printf("%d", c);
}
```

정답: 505

해설:
1부터 2023까지 중에서 4의 배수의 개수를 세는 코드입니다.

- i % 4 == 0 은 "i를 4로 나눈 나머지가 0", 즉 i가 4의 배수라는 뜻입니다.
- 4의 배수: 4, 8, 12, ..., 2020
- 개수 = 2020 / 4 = 505개

따라서 c는 505가 됩니다.

핵심: "N 이하의 K의 배수 개수 = N / K (정수 나눗셈)"입니다. 2023 / 4 = 505 (나머지 3은 버려짐).

8번

다음 내용에 알맞는 답을 작성하시오.

- 워터마크 삭제 등 소프트웨어가 불법으로 변경되었을 경우 정상 수행되지 않게 하는 기법
- 변조검증코드(tamper-proofing code)를 삽입하여 변경 탐지 및 실행 차단
- 소프트웨어 위변조 방지 역공학 기술의 일종

정답: 템퍼프루핑(Tamper-proofing)

해설:

템퍼프루핑(Tamper-proofing)은 "조작 방지"라는 뜻으로, 소프트웨어가 **불법으로 변조되면 정상 작동하지 않게** 만드는 보안 기법입니다.

- "Tamper"는 "함부로 손대다, 조작하다"라는 뜻입니다.
- 프로그램 안에 **변조 검증 코드**를 심어두고, 누군가 코드를 몰래 바꾸면(예: 워터마크 제거, 라이선스 우회) 이를 감지해 실행을 멈춥니다.

함께 알아둘 소프트웨어 저작권/보안 보호 기술:

- **디지털 워터마킹**: 저작권 정보를 보이지 않게 삽입
- **핑거프린팅**: 구매자별 고유 정보 삽입(불법 유포자 추적)
- **템퍼프루핑**: 변조 시 작동 차단

9번

다음은 C언어 문제이다. 알맞은 출력값을 작성하시오.

```
#include <stdio.h>
#define MAX_SIZE 10

int isWhat[MAX_SIZE];
int point = -1;

int isEmpty() {
    if (point == -1) return 1;
    return 0;
}

int isFull() {
    if (point == 10) return 1;
    return 0;
}

void into(int num) {
    if (point >= 10) printf("Full");
    else isWhat[++point] = num;
}

int take() {
    if (isEmpty() == 1) printf("Empty");
    else return isWhat[point--];
    return 0;
}

int main(int argc, char const *argv[]){
    int e;
    into(5); into(2);

    while(!isEmpty()){
        printf("%d", take());
        into(4); into(1); printf("%d", take());
        into(3); printf("%d", take()); printf("%d", take());
        into(6); printf("%d", take()); printf("%d", take());
    }

    return 0;
}
```

정답: 213465

해설:

이 코드는 **스택(Stack)** 자료구조를 흉내 낸 것입니다. 스택은 **LIFO(Last In First Out, 나중에 넣은 게 먼저 나옴)** 구조입니다. 접시를 쌓았다가 위에서부터 빼는 것과 같습니다.

- `into(num)` : 값을 스택에 **넣기(push)** → `isWhat[++point]`
- `take()` : 스택에서 값을 **빼기(pop)** → `isWhat[point--]` (가장 위, 즉 마지막에 넣은 값)

흐름 추적 (스택 상태와 출력):

1. `into(5)` `into(2)` → 스택: [5, 2]
2. `take()` → 마지막 2 빼냄 → 출력 **2** / 스택: [5]
3. `into(4)` `into(1)` → 스택: [5, 4, 1] / `take()` → 1 빼냄 → 출력 **1** / 스택: [5, 4]
4. `into(3)` → 스택: [5, 4, 3] / `take()` → 3 → 출력 **3** / `take()` → 4 → 출력 **4** / 스택: [5]
5. `into(6)` → 스택: [5, 6] / `take()` → 6 → 출력 **6** / `take()` → 5 → 출력 **5** / 스택: []
6. 스택이 비었으므로 while 종료

출력 순서: 2 → 1 → 3 → 4 → 6 → 5 = **213465**

핵심: 스택은 항상 "맨 위(가장 최근에 넣은 것)"부터 나온다는 LIFO 원리를 손으로 그려가며 따라가세요.

10번

데이터베이스 설계 순서에 관한 내용이다. 보기를 이용하여 괄호 안에 알맞은 내용을 작성하시오. (※ 원문에 이미지 포함)

보기: 구현, 요구조건 분석, 개념적 설계, 물리적 설계, 논리적 설계

정답: 요구조건 분석 → 개념적 설계 → 논리적 설계 → 물리적 설계 → 구현

해설:

데이터베이스 **설계 5단계**는 시험에 매우 자주 나오므로 순서를 통째로 외워야 합니다.

1. **요구조건 분석**: 사용자가 무엇을 원하는지 파악하고 정리합니다.
2. **개념적 설계**: 현실 세계를 추상화하여 **E-R 다이어그램**(개체-관계 모델)을 그립니다. DBMS에 독립적입니다.
3. **논리적 설계**: 개념 모델을 실제 DBMS가 처리할 수 있는 **테이블(릴레이션) 구조**로 바꿉니다. 정규화도 이 단계에서 합니다.
4. **물리적 설계**: 저장장치에 어떻게 저장할지(인덱스, 접근 방법 등)를 정합니다.
5. **구현**: 실제로 DDL(테이블 생성 등)로 데이터베이스를 만듭니다.

암기 팁: "**요-개-논-물-구**" (요구분석-개념-논리-물리-구현). "개념은 ER, 논리는 테이블"이라는 핵심도 같이 외우세요.

11번

다음은 디자인 패턴에 관한 문제이다. 보기에서 알맞는 답을 작성하시오.

1. 생성자가 여러 차례 호출되더라도 실제 생성되는 객체는 하나이고, 이후 생성자는 최초 객체를 리턴. DBCP 같은 상황에서 사용.
2. 호스트 객체의 내부 상태에 접근하여 연산을 추가. 합성 구조 원소들과 상호작용하며 기존 코드를 변경하지 않고 새 기능 추가.

보기: (생성) Singleton, Factory Method, Builder / (구조) Adapter, Bridge, Decorator / (행위) Observer, Strategy, Visitor

정답: 1. Singleton / 2. Visitor

해설:

GoF 디자인 패턴 문제입니다.

1. **싱글톤(Singleton)** — 생성 패턴
 - 아무리 여러 번 객체를 만들려 해도 **실제로는 단 하나의 객체만** 만들어지고, 그 하나를 모두가 공유합니다.
 - DB 연결 풀(DBCP), 설정 관리자처럼 "전역에서 하나만 있으면 되는 것"에 씁니다.
 - 키워드: "**객체는 하나만**".
2. **비지터(Visitor)** — 행위 패턴
 - 객체의 구조는 그대로 두고, **새로운 기능(연산)을 외부에서 추가**합니다.
 - 기존 클래스 코드를 수정하지 않고도 기능을 확장할 수 있습니다.

- 키워드: "기존 코드 변경 없이 새 기능 추가".

참고 - GoF 패턴 3분류:

- **생성**: Singleton, Factory Method, Abstract Factory, Builder, Prototype
- **구조**: Adapter, Bridge, Composite, Decorator, Facade, Proxy, Flyweight
- **행위**: Observer, Strategy, Visitor, Iterator, Command, State 등

12번

다음 내용에서 설명하는 문제에 대해 보기에 알맞은 답을 골라 작성하시오.

- (1) Code는 1비트 에러를 정정할 수 있는 오류정정부호. Bell 연구소의 Hamming이 고안. 선형블록부호·순회부호에 속함.
- (2) 송신측이 부가적 정보를 첨가하여 전송하고 수신측이 이를 이용해 에러검출 및 정정.
- (3) 오류 발생 시 송신측에 재전송을 요구하는 방식. Parity, CRC, 블록 합 검사 등.
- (4) 데이터가 이동/전송될 때 유실·손상 여부를 점검하는 기술.
- (5) 데이터 전송 시 오류 확인을 위한 체크값을 결정하는 방식.

보기: EAC, FEC, hamming, CRC, PDS, parity, BEC

정답: 1. hamming / 2. FEC / 3. BEC / 4. parity / 5. CRC

해설:

오류 검출·정정 관련 용어 문제입니다. 오류 처리 방식은 크게 두 갈래입니다.

- **FEC(Forward Error Correction, 전진/순방향 오류정정)**: 수신측이 스스로 오류를 정정합니다. 재전송이 필요 없어 빠르지만, 여분 정보가 많이 붙습니다.
대표: **해밍코드(Hamming)**.
- **BEC(Backward Error Correction, 후진 오류정정)**: 오류가 발견되면 송신측에 재전송을 요청합니다. 대표: Parity, CRC.

각 답:

1. **hamming(해밍코드)**: 검사 비트를 추가해 **1비트 오류를 정정**까지 할 수 있는 코드.
2. **FEC**: 송신측이 정보를 덧붙여 수신측이 직접 정정.
3. **BEC**: 오류 시 재전송 요구.
4. **parity(패리티)**: 1의 개수를 짝/홀로 맞춰 오류를 **검출**하는 가장 간단한 방법.
5. **CRC(순환중복검사)**: 다항식 나눗셈으로 체크값을 만들어 오류를 검출. 강력하고 널리 쓰임.

암기 팁: "**정정 = FEC(해밍), 재전송 = BEC(Parity/CRC)**".

13번

다음은 HDLC 프로토콜에 대한 설명이다. 보기에서 알맞은 답을 골라 작성하시오.

- (1) Seq, Next, P/F 필드를 가짐. 맨 처음 비트 0. 송신용 순서번호를 가지는 프레임.
- (2) 맨 앞 필드 1로 정보 프레임이 아님을 표시, 다음 비트 0. 응답 기능 수행, Seq 없이 Next만 존재.
- (3) 순서 번호가 없는 프레임. 첫·두 번째 비트 모두 1. Type 2비트 + Modifier 3비트로 종류 구분.
- (4) 두 호스트 모두 혼합국으로 동작. 양쪽에서 명령·응답 전송 가능.
- (5) 불균형 모드로 주국 허락 없이 종국에서 데이터 전송 가능.

보기: ㄱ. 연결제어 ㄴ. 감독 ㄷ. 정보 ㄹ. 양방향 응답 ㄴ. 익명 ㄷ. 비번호 ㄴ. 릴레이 ㄹ. 동기균형 ㄷ. 동기응답 ㄴ. 비동기균형 ㄷ. 비동기응답

정답: 1. ㄷ / 2. ㄴ / 3. ㄷ / 4. ㄴ / 5. ㄷ

해설:

HDLC(High-level Data Link Control)는 데이터링크 계층의 프로토콜로, **프레임 3종류**와 **동작 모드 3종류**가 시험에 나옵니다.

프레임 3종류 (맨 앞 비트로 구분):

1. **정보 프레임(I-프레임)**: 맨 앞 비트 **0**. 실제 사용자 데이터를 전송. 순서번호(Seq) 보유.
2. **감독 프레임(S-프레임)**: 맨 앞 비트 **10**. 데이터 없이 **흐름 제어·오류 제어(응답)** 담당. Seq 없음, Next만 있음.
3. **비번호 프레임(U-프레임)**: 맨 앞 비트 **11**. 순서번호 없음. 링크 **설정/해제, 모드 설정** 등 제어용.

동작 모드 3종류:

- **정규 응답 모드(NRM)**: 불균형. 종국은 주국 허락이 있어야 전송.
- **비동기 응답 모드(ARM)**: 불균형. **종국이 주국 허락 없이도** 전송 가능. → (5)

- 비동기 균형 모드(ABM): 두 국 모두 대등(혼합국)하게 명령·응답 가능. → (4)

암기 팁: "정보=0(데이터), 감독=10(제어/응답), 비번호=11(링크관리)".

14번

다음은 자바에 대한 문제이다. 알맞은 출력값을 작성하시오.

```
public class Main {
    public static void main(String[] args) {
        String str1 = "Programming";
        String str2 = "Programming";
        String str3 = new String("Programming");

        println(str1 == str2);
        println(str1 == str3);
        println(str1.equals(str3));
        print(str2.equals(str3));
    }
}
```

정답: true / false / true / true

해설:
자바 String 비교에서 `==` 와 `.equals()` 의 차이를 묻는 매우 중요한 문제입니다.

- `==` : 두 변수가 같은 객체(같은 메모리 주소)를 가리키는지 비교합니다. (주소 비교)
- `.equals()` : 두 문자열의 내용(글자)이 같은지 비교합니다. (값 비교)

핵심 개념 - 문자열 상수 풀(String Constant Pool):
- "Programming" 처럼 큰따옴표로 직접 쓴 문자열은 풀에 저장되어 재사용됩니다. 같은 글자면 같은 주소를 공유합니다.
- `new String("Programming")` 은 무조건 새 객체를 따로 만듭니다(다른 주소).

분석:
- `str1 == str2` → 둘 다 풀의 같은 "Programming" → 같은 주소 → **true**
- `str1 == str3` → str3는 new로 만든 새 객체 → 주소 다름 → **false**
- `str1.equals(str3)` → 내용은 둘 다 "Programming" → **true**
- `str2.equals(str3)` → 내용 같음 → **true**

핵심 암기: "`==` 는 주소, `equals` 는 값". 문자열 내용 비교는 항상 `.equals()` 를 써야 안전합니다.

15번

다음 보기는 암호화 알고리즘에 대한 내용이다. 대칭키와 비대칭키에 해당하는 보기를 작성하시오.

보기: DES, RSA, AES, ECC, ARIA, SEED

정답:
- 대칭키: DES, AES, ARIA, SEED
- 비대칭키: RSA, ECC

해설:
암호화 방식은 키 사용 방법에 따라 두 가지로 나뉩니다.

- 대칭키(비밀키) 암호화: 암호화와 복호화에 같은 키 하나를 사용합니다. 빠르지만, 키를 안전하게 전달하기 어렵습니다.
- 종류: DES, 3DES, AES, ARIA, SEED, IDEA, RC4 등
- 비대칭키(공개키) 암호화: 공개키와 개인키 두 개를 사용합니다. 느리지만 키 분배가 쉽고 안전합니다.
- 종류: RSA, ECC, DSA, Diffie-Hellman, ElGamal 등

암기 팁: 비대칭키는 RSA, ECC, DSA(영문 약자 위주)만 따로 외우고, 나머지는 대부분 대칭키라고 기억하면 편합니다. (특히 우리나라 알고리즘 ARIA, SEED는 대칭키)

16번

다음 괄호 안에 알맞는 답을 작성하시오.

- ()란 임의의 크기를 가진 데이터(Key)를 고정된 크기의 데이터(Value)로 변화시켜 저장하는 것
- 키에 대한 () 값을 사용하여 값을 저장하고, 키-값 쌍의 개수에 따라 동적으로 크기 증가
- () 값 자체를 index로 사용하여 평균 시간복잡도 O(1)
- 큰 파일에서 중복 레코드를 찾을 수 있어 검색 속도 가속

정답: 해시 (해싱, hash)

해설:

해시(Hash)는 임의 크기의 데이터를 고정된 크기의 값으로 바꾸는 기술입니다.

- 해시 함수에 키(Key)를 넣으면 해시값(주소)이 나옵니다. 이 해시값을 인덱스로 삼아 데이터를 저장합니다.
- 장점: 키를 함수에 넣으면 바로 저장 위치를 계산할 수 있어, 평균 검색 속도가 **O(1)**(매우 빠름)입니다. 처음부터 하나씩 찾을 필요가 없습니다.
- 해시 테이블은 이 원리로 만든 자료구조입니다.

비유: 도서관에서 책 제목(키)을 함수에 넣으면 "3층 A-12 칸"(해시값)이 바로 나와 즉시 찾는 것과 같습니다.

참고: 서로 다른 키가 같은 해시값을 갖는 것을 충돌(Collision)이라 하며, 이를 해결하는 방법(체이닝, 개방 주소법 등)도 함께 알아두면 좋습니다.

17번

다음 보기의 SQL문에서 괄호 안에 알맞는 단어를 작성하시오.

```
DROP VIEW 학생 (      )
```

- 학생 테이블을 참조하는 다른 VIEW나 제약 조건까지 모두 삭제되어야 한다.

정답: cascade (대소문자 무관)

해설:

DROP 은 데이터베이스 객체(테이블, 뷰 등)를 완전히 삭제하는 명령입니다. 여기서 **CASCADE 옵션**이 핵심입니다.

- **CASCADE(연쇄)**: 삭제하려는 대상을 참조하는 다른 객체들까지 함께(연쇄적으로) 삭제합니다. "줄줄이 다 지운다"는 의미입니다.
- **RESTRICT(제한)**: 참조하는 다른 객체가 하나라도 있으면 삭제를 거부합니다. 안전 장치 역할입니다.

지문에서 "참조하는 다른 VIEW나 제약 조건까지 모두 삭제"라고 했으므로 → **CASCADE**.

암기 팁: "**CASCADE = 연쇄 삭제(다 지움)**, **RESTRICT = 참조 있으면 거부(못 지움)**".

18번

다음 코드는 선택정렬 구현에 관한 문제이다. 오름차순으로 정렬할 경우 빈칸에 알맞는 연산자를 보기에서 골라 작성하시오.

```
#include <stdio.h>

int main() {
    int E[] = {64, 25, 12, 22, 11};
    int n = sizeof(E) / sizeof(E[0]);
    int i = 0;
    do {
        int j = i + 1;
        do {
            if (E[i] (      ) E[j]) {
                int tmp = E[i];
                E[i] = E[j];
                E[j] = tmp;
            }
            j++;
        } while (j < n);
        i++;
    } while (i < n-1);
}
```

```
for(int i=0; i<=4; i++)
    printf("%d ", E[i]);
}
```

보기: <, <=, >, >=, /, %

정답: >

해설:

이 코드는 **선택 정렬(Selection Sort)**의 변형으로, 앞쪽 자리(E[i])에 더 작은 값을 모아 **오름차순**(작은 값 → 큰 값)으로 정렬합니다.

비교 부분 if (E[i] (?) E[j]) 이 참이면 두 값을 교환(swap)합니다.

- **오름차순**으로 만들려면 앞자리(E[i])에 작은 값이 와야 합니다.
- 만약 E[i] 가 E[j] 보다 **크다(>)**면, 앞에 큰 값이 잘못 놓인 것이므로 교환해서 작은 값을 앞으로 보냅니다.
- 따라서 빈칸은 > 입니다.

기억법: "**오름차순 → 앞이 더 크면(>) 교환**", "**내림차순 → 앞이 더 작으면(<) 교환**". 부등호 방향이 정렬 방향을 결정합니다.

19번

다음 파이썬 코드에서 알맞는 출력값을 작성하시오.

```
a = "engineer information processing"
b = a[:3]
c = a[4:6]
d = a[28:]
e = b + c + d
print(e)
```

정답: engneing

해설:

파이썬의 **슬라이싱(slicing)**을 묻는 문제입니다. 문자열 [시작:끝] 은 시작 인덱스부터 **끝 인덱스 바로 앞까지** 잘라냅니다. (끝 인덱스는 포함 안 됨)

먼저 문자열 인덱스를 확인합니다 (0부터 시작):

```
e n g i n e e r   i n f o r m a t i o n       p r o c e s s i n g
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
```

- b = a[:3] → 인덱스 0,1,2 → "eng"
- c = a[4:6] → 인덱스 4,5 → "ne"
- d = a[28:] → 인덱스 28부터 끝까지 → "ing"
- e = b + c + d = "eng" + "ne" + "ing" = "**engneing**"

핵심: 슬라이싱은 "**시작 포함, 끝 미포함**"이라는 규칙과, 인덱스가 **0부터** 시작한다는 점을 정확히 세는 것이 관건입니다.

20번

다음 설명에 대한 알맞는 답을 작성하시오.

1. 하향식 테스트 시 상위 모듈은 존재하나 하위 모듈이 없는 경우 임시 제공되는 모듈. 단순히 값만 넘겨주는 가상의 클라이언트.
2. 상향식 테스트 시 상위 모듈 없이 하위 모듈이 존재할 때 자료 입출력을 제어하는 모듈. 간단한 기능만 하는 가상의 서버.

정답: 1. 스텝 / 2. 드라이버

해설:

통합 테스트에서 아직 안 만들어진 모듈을 대신할 **가짜(더미) 모듈**에 관한 문제입니다.

- **스텝(Stub): 하향식(위→아래)** 테스트에서 사용. 상위 모듈은 있는데 **하위 모듈이 아직 없을 때**, 그 하위 모듈을 흉내 내는 가짜입니다. 단순히 정해진 값만 돌려줍니다.
- **드라이버(Driver): 상향식(아래→위)** 테스트에서 사용. 하위 모듈은 있는데 **상위 모듈이 아직 없을 때**, 하위 모듈을 호출해주는 가짜 상위 모듈입니다.

암기 팁: "하향식 → 스텝(아래 대신), 상향식 → 드라이버(위 대신)".

- 헛갈릴 때: 하향식의 '하'와 스텝을 묶고, 상향식의 드라이버(운전자가 위에서 끌고 감)로 연상하면 좋습니다.